PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Taller investigativo arquitecturas de software

Presentado a: Yoly Alexandra Guevara Muñoz

Por Aprendiz: Miguel Angel Rivera

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 01 de mes 10 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1.

1.1 Para qué sirven Node.js y npm

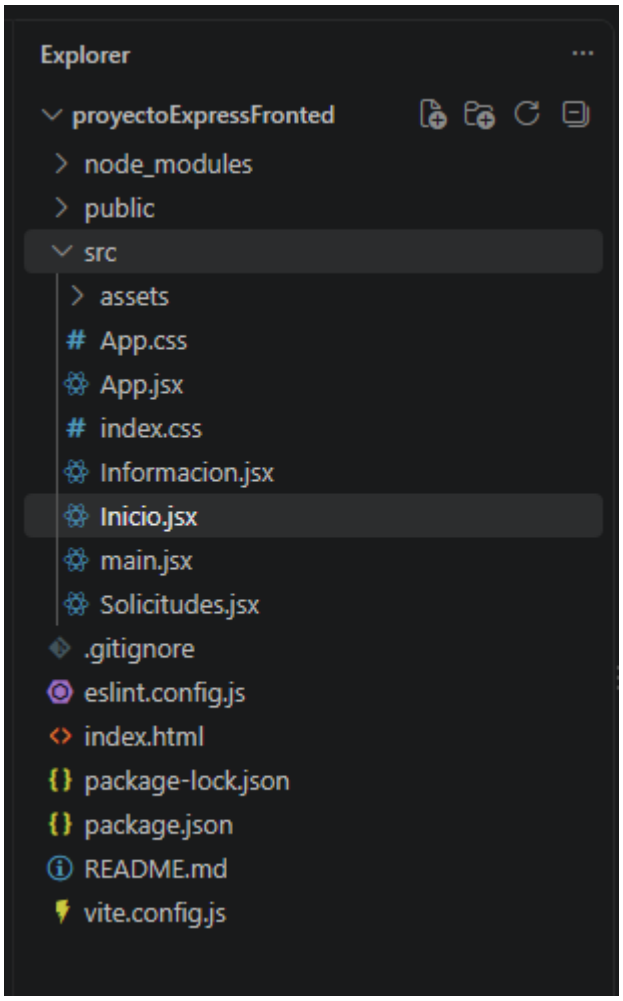
Node.js permite ejecutar JavaScript fuera del navegador, en mi computador. Lo necesité para correr el servidor Express y también para Vite, que es la herramienta que levanta el proyecto de React. npm es el administrador de paquetes de Node: con él descargo librerías (React, Vite, cors) y las registra en el archivo package.json.

1.2 creación del proyecto con vite:

Creé el proyecto con estos comandos:
npm create vite@latest proyectoExpressFronted
cd proyectoExpressFrontend
npm install
npm run dev
El primero genera la estructura del proyecto con la plantilla de React en JavaScript. npm install descarga las dependencias a la carpeta node_modules. npm run dev inicia el servidor de desarrollo de Vite, que muestra la aplicación en http://localhost:5173 y la actualiza cada vez que guardo un cambio.

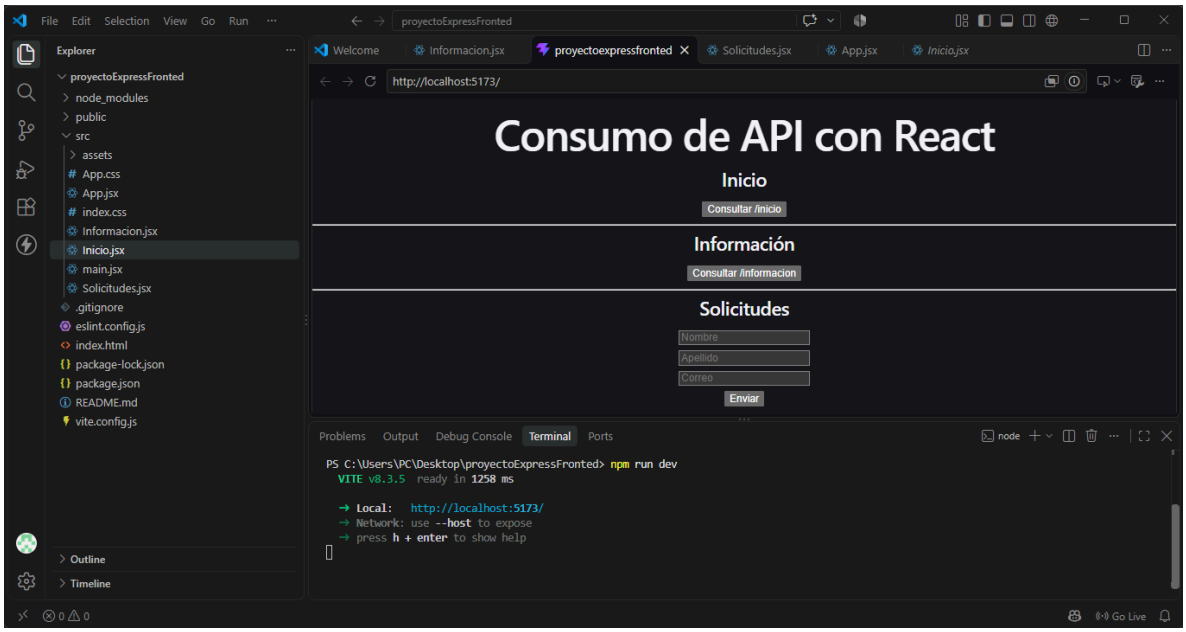
1.3

Archivo	**Función**
src/App.jsx	**Es el componente principal. Reúne los tres componentes de la actividad.**
src/Inicio.jsx	**Componente que consulta GET /inicio.**
src/Informacion.jsx	**Componente que consulta GET /informacion.**
src/Solicitudes.jsx	**Componente con el formulario que envía POST /solicitudes.**



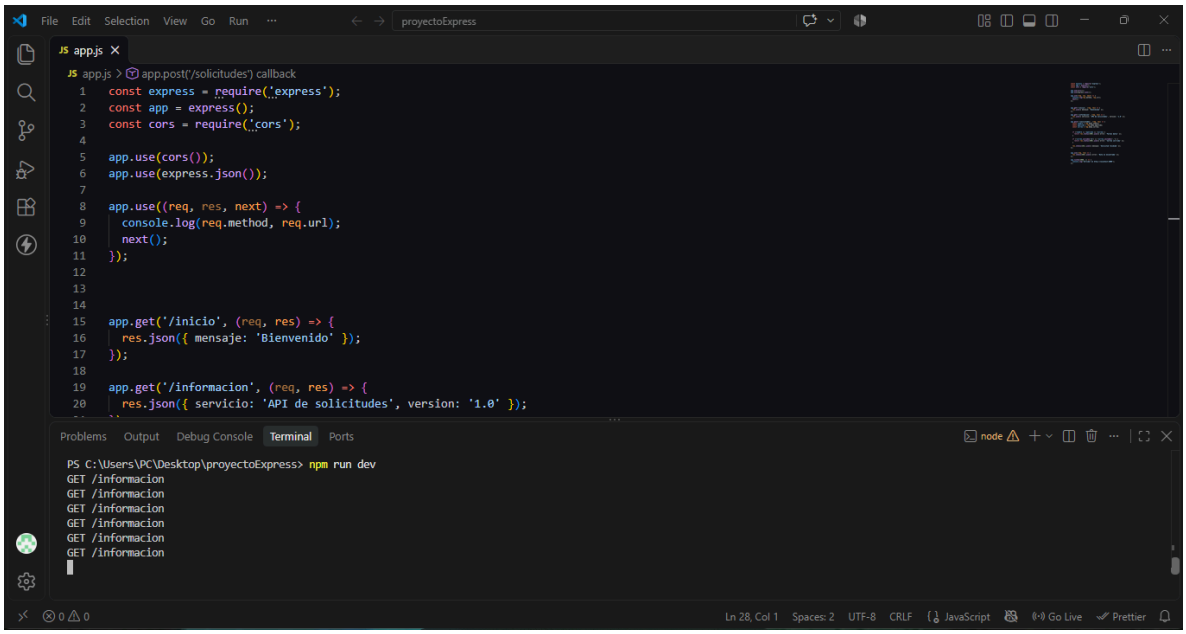
1.4

Dentro de la carpeta proyectoExpressFronted ejecuto npm run dev y abro en el navegador la dirección que muestra la terminal (http://localhost:5173). Dejo esa terminal abierta mientras trabajo, porque si la cierro el servidor se detiene.



1.5

Antes de hacer pruebas inicio la API en su carpeta con npm run dev y compruebo que la terminal muestre el mensaje Servidor en http://localhost:3000. Después envío una petición GET a http://localhost:3000/inicio desde Thunder Client y confirmó que responde con código 200 y el mensaje en JSON.



2.

2.1 componentes de inicio:

- Responsabilidad: consultar el mensaje de bienvenida de la API y mostrarlo.
- Endpoint: /inicio.
- Método HTTP: GET.
- Cómo se ejecuta la petición: al hacer clic en el botón Consultar /inicio
- Cómo recibe y procesa la respuesta: con await espero la respuesta, la convierto con response.json() y reviso response.ok. Si es correcta, guardo data.mensaje en el estado.
- Cómo muestra la información: al cambiar el estado, React vuelve a dibujar el componente y el mensaje aparece en un párrafo. Mientras espera muestra Cargando...
- Errores y manejo: si el servidor está apagado, fetch() falla y el catch muestra No se pudo conectar con el servidor. Si la API responde con un código de error, muestra el código recibido.

2.2 componentes de información:

- Responsabilidad: consultar la información del servicio y mostrarla.
- Endpoint: /informacion.

- Método HTTP: GET.
- Cómo se ejecuta la petición: con el botón Consultar /informacion.
- Cómo recibe y procesa la respuesta: convierto la respuesta con response.json() y guardo el objeto completo en el estado info. El objeto tiene las propiedades servicio y version.
- Cómo muestra la información: el componente lee info.servicio e info.version y los escribe en pantalla. React usa el estado para decidir qué dibujar: mientras info está vacío no muestra nada, y cuando llega la respuesta se actualiza la interfaz.
- Errores y manejo: igual que en Inicio, con mensaje de conexión y mensaje de código de error.

2.3 componentes de solicitudes:

- Responsabilidad: capturar nombre, apellido y correo, enviarlos a la API y mostrar el resultado.
- Endpoint: /solicitudes.
- Método HTTP: POST.
- Cómo se ejecuta la petición: al enviar el formulario, el evento onSubmit llama a enviarFormulario, que usa e.preventDefault() para que la página no se recargue y luego hace fetch() con method POST, la cabecera Content-Type y el cuerpo en JSON.
- Cómo recibe y procesa la respuesta: convierto la respuesta con response.json(). Si response.ok es verdadero, guardo data.mensaje y limpio los campos. Si no, guardo data.error, que trae el mensaje de validación de Express (por ejemplo Faltan datos o Correo inválido).
- Cómo muestra la información: los mensajes de éxito, error y carga se muestran debajo del formulario.
- Errores y manejo: hay dos tipos. Si la API rechaza los datos (código 400), muestro el mensaje que envía el backend. Si no se puede conectar con el servidor, el catch muestra No se pudo conectar con el servidor.

3.

fetch(). Es la función del navegador que hace la petición HTTP. Recibe la dirección y, para POST, un objeto con el método, las cabeceras y el cuerpo. Para los GET basta con la dirección, porque GET es el método por defecto.

async/await. Una petición tarda un tiempo en responder. Con async declaro que la función es asíncrona, y con await le digo que espere el resultado antes de seguir a la siguiente línea, sin congelar la página.

response.json(). La respuesta del servidor llega como texto. Este método lo convierte en un objeto de JavaScript, para poder usar data.mensaje o data.error.

JSON.stringify(). Hace lo contrario: convierte el objeto con nombre, apellido y correo en texto JSON, que es lo único que se puede enviar en el cuerpo de la petición. La cabecera Content-Type: application/json le avisa a Express que ese texto es JSON, y así el middleware express.json() lo convierte en req.body.

useState. Guarda los datos que cambian: lo que el usuario escribe como nombre, apellido, correo y lo que se muestra como mensaje, error, cargando. Cuando llamo a una función como setMensaje(), React vuelve a dibujar el componente con el valor nuevo. Los inputs usan value={nombre} y onChange={(e) => setNombre(e.target.value)} para que el estado siempre tenga lo que se escribió.

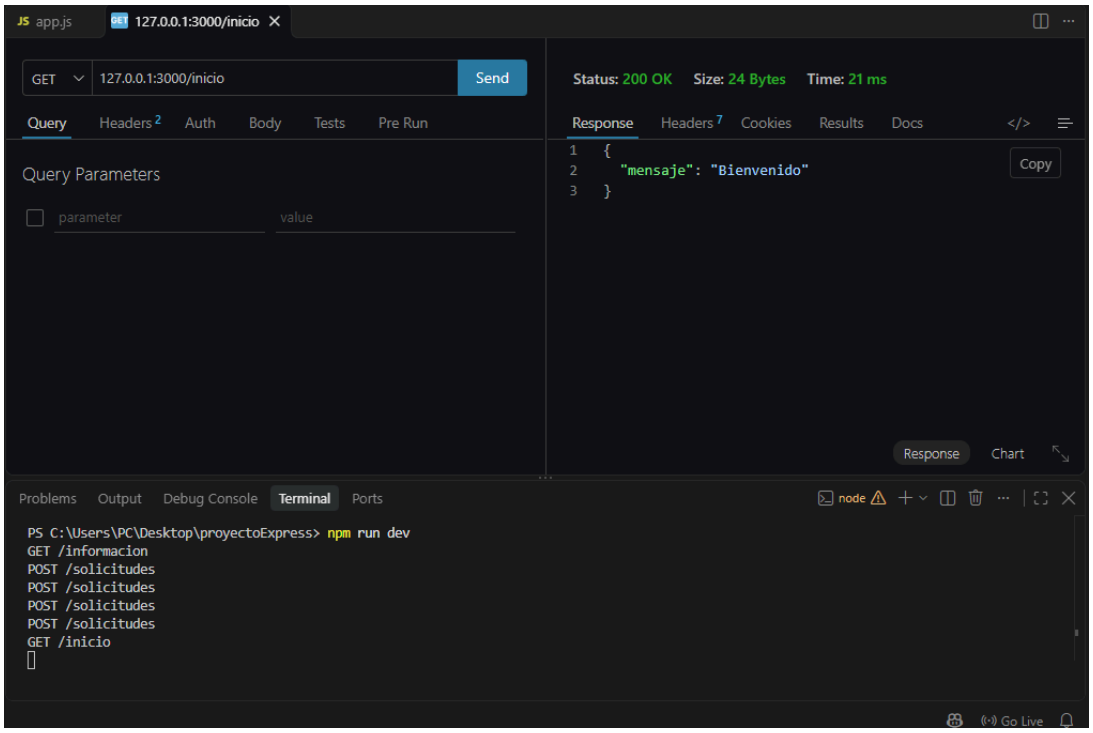
Eventos. onClick ejecuta la consulta cuando el usuario presiona un botón. onSubmit ejecuta el envío del formulario, y e.preventDefault() evita que el navegador recargue la página, que es su comportamiento normal al enviar un formulario. onChange actualiza el estado en cada letra escrita.

Manejo de errores. Hay que separar dos situaciones. Si el servidor responde con 400 o 404, fetch() no falla: entrega la respuesta y por eso reviso response.ok (verdadero solo para códigos 200 a 299) y muestro el error que envió el backend. Si el servidor está apagado o no hay conexión, fetch() falla y el control pasa al catch, donde muestro el mensaje de error de conexión. No asumo que una petición fue exitosa solo porque fetch() devolvió algo.

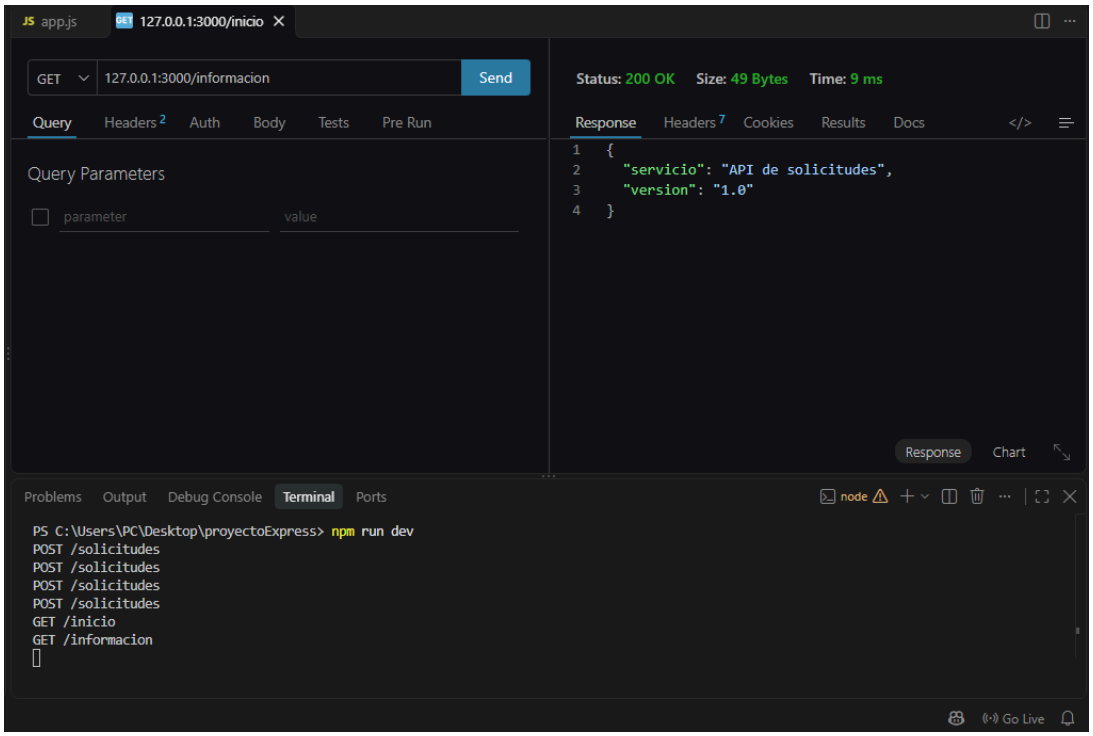
4.

Consultar /inicio

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

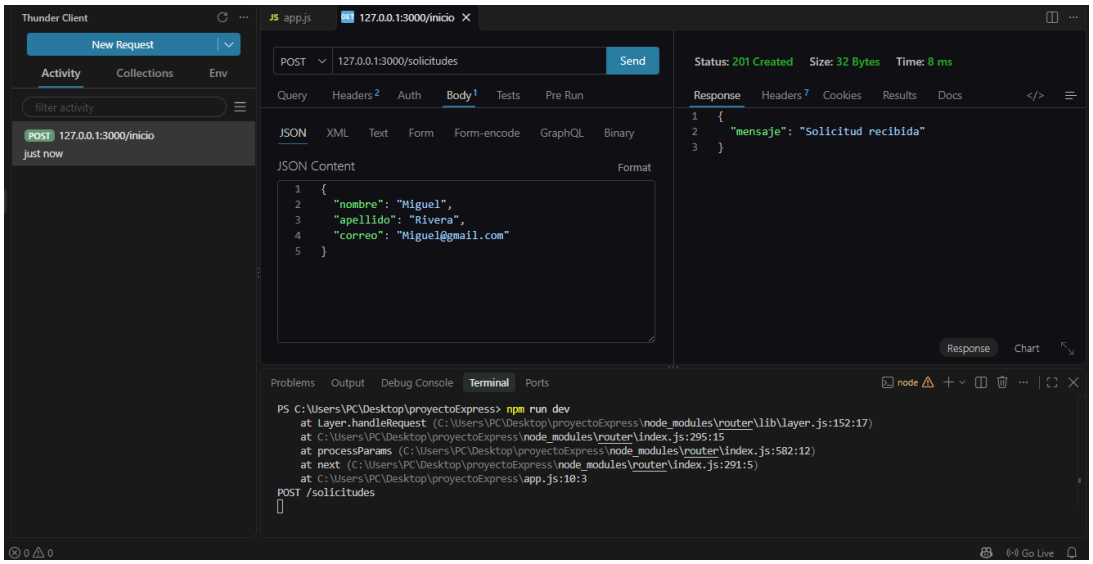


Consultar /informacion

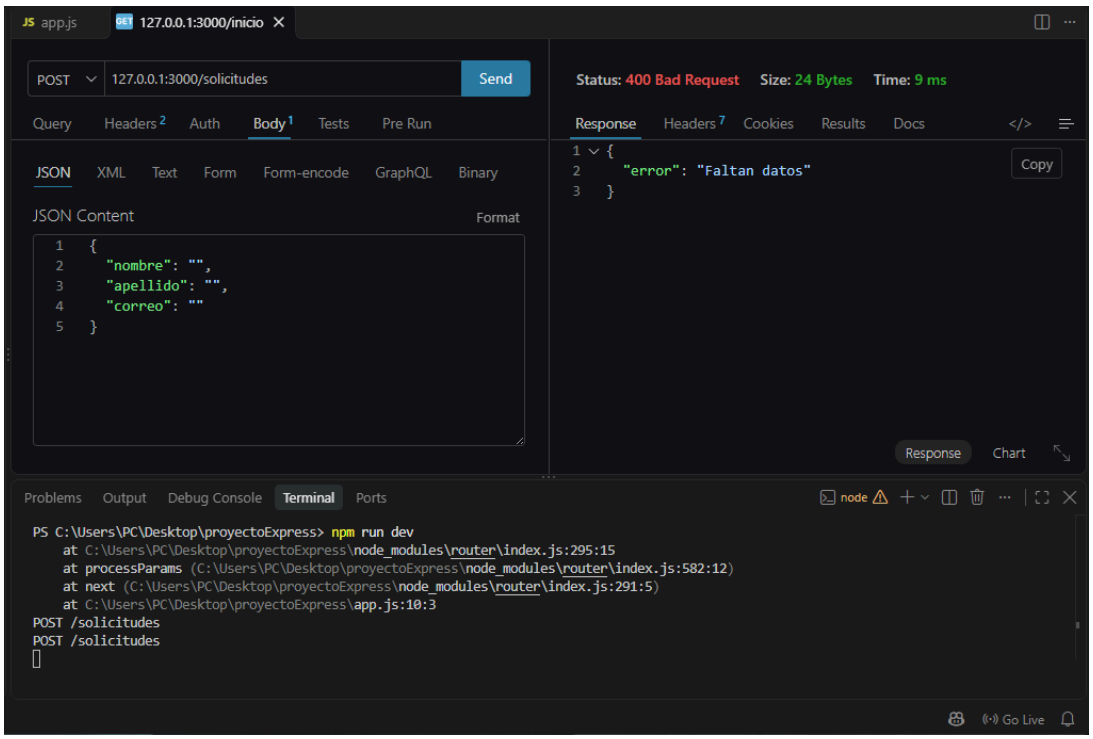


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

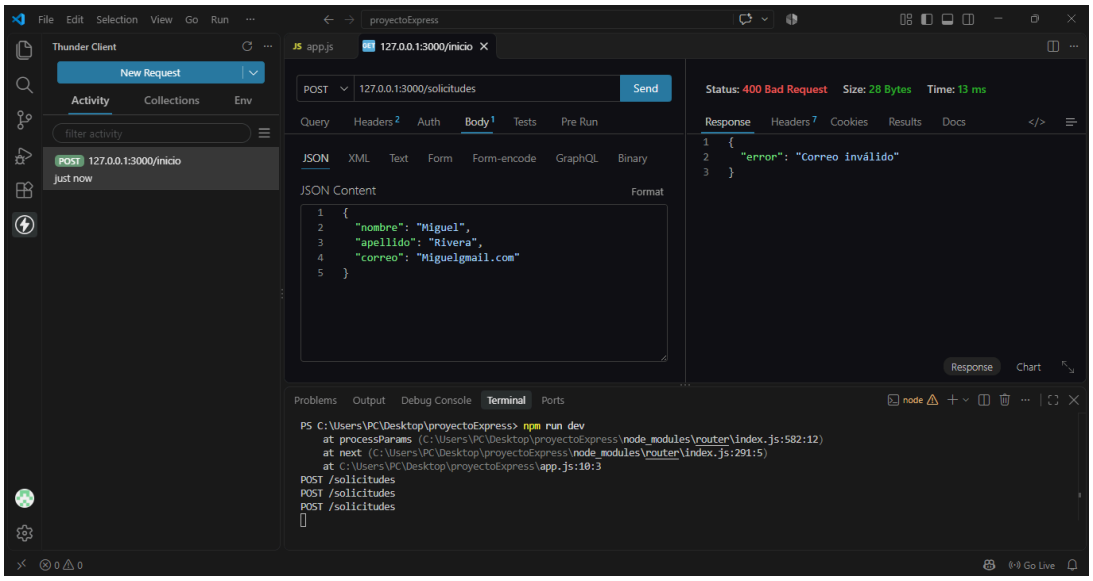
Formulario completo



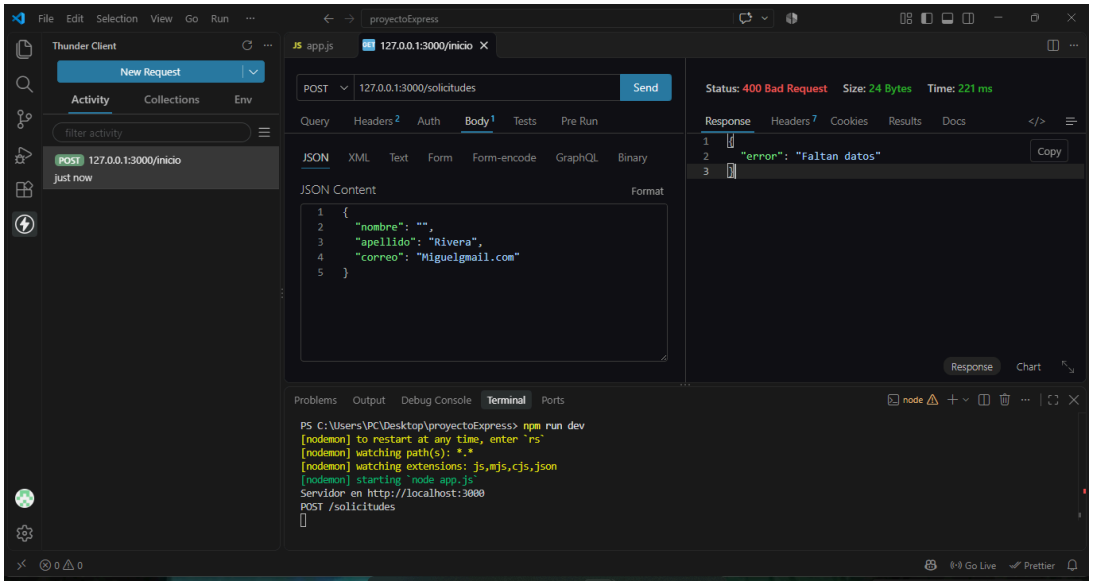
Campos vacíos



Correo inválido



Datos inválidos



5.

Un problema que salió es que al presionar Consultar /inicio, la aplicación mostraba Cargando y nunca aparecía el mensaje ni un error.

lo identifiqué al abrír la consola del navegador y apareció un error de CORS: el navegador bloqueó la petición porque React (puerto 5173) y Express (puerto 3000) son orígenes distintos y la API no lo permitía.

Solución: en la carpeta de la API instalé la librería cors con npm install cors y agregué const cors = require('cors') y app.use(cors()) en app.js, antes de express.json(). Reinicié el servidor.

Lo verifique volviendo a presionar el botón y apareció el mensaje de la API. En la pestaña Network, la respuesta ya traía la cabecera Access-Control-Allow-Origin.

6. **Investigación sobre consumo de API:**

   1. ¿Qué es una API y para qué sirve en una aplicación web? Es un conjunto de rutas y reglas con las que un programa le pide datos o acciones a otro. En una aplicación web sirve para que el frontend consulte o guarde información en el servidor sin conocer cómo funciona por dentro.

   2. ¿Diferencia entre frontend y backend? El frontend es la parte que el usuario ve y usa en el navegador. El backend corre en el servidor, procesa las peticiones, valida datos y responde.

   3. ¿Qué significa consumir una API? Es hacer peticiones a sus rutas desde otra aplicación y usar las respuestas. Mi React consume la API de Express al pedir /inicio, /informacion y /solicitudes.

   4. ¿Qué función cumple fetch()? Es la función del navegador que envía peticiones HTTP a una dirección y devuelve una promesa con la respuesta.

   5. ¿Por qué se usan async y await en una petición HTTP? Porque la respuesta tarda. Con await el código espera el resultado antes de usarlo, y con async se declara que la función puede esperar, sin bloquear la página.

   6. ¿Qué es una promesa en JavaScript? Es un objeto que representa un resultado que todavía no está listo: puede terminar bien o con fallos.

   7. ¿Para qué sirve response.json()? Para leer el body de la respuesta, que llega como texto, y convertirlo en un objeto de JavaScript con el que se puede trabajar.

   8. ¿Diferencia entre GET y POST? GET sirve para pedir información y no lleva body. POST sirve para enviar datos al servidor, que van en el cuerpo de la petición para crear algo.

   9. ¿Qué función cumplen las cabeceras HTTP y por qué se usa Content-Type: application/json? Las cabeceras envían información adicional sobre la petición. Content-Type le dice al servidor en qué formato

viene el cuerpo; con application/json, Express sabe que debe interpretarlo como JSON.

10. ¿Por qué se usa JSON.stringify() al enviar datos en un POST? Porque el cuerpo de una petición debe ser texto, no un objeto de JavaScript. stringify convierte el objeto en texto JSON.

11. ¿Diferencia entre un error de conexión y una respuesta HTTP con código 400? En el error de conexión la petición nunca llega o no hay respuesta y fetch() falla. Con un 400 sí hubo respuesta, el servidor recibió la petición y rechazó los datos, y fetch() no falla.

12. ¿Qué función cumplen useState, onClick y onSubmit? useState guarda datos que cambian y hace que React vuelva a dibujar el componente cuando cambian. onClick ejecuta una función al hacer clic. onSubmit ejecuta una función al enviar un formulario.

13. ¿Qué es CORS y por qué puede afectar la comunicación entre React y Express? Es una regla de seguridad del navegador que bloquea peticiones a un origen distinto si el servidor no lo autoriza. React y Express son orígenes distintos, así que Express debe permitirlo, por ejemplo con la librería cors.

14. ¿Qué ocurre si el frontend envía propiedades diferentes a las que espera el backend? El backend no encuentra los campos que necesita (nombre, apellido, correo) y los toma como faltantes, por lo que rechaza la petición con un 400. Por eso el frontend debe usar los mismos nombres que define la API.

15. ¿Por qué no se debe considerar exitosa toda petición solo porque fetch() devolvió una respuesta? Porque fetch() solo falla cuando no hay conexión. Un 400, 404 o 500 también llegan como respuesta, y hay que revisar response.ok o el código de estado para saber si realmente salió bien.